

Structurally Corrected Invariant-Preserving Neural Networks for Conservative Dynamical Systems: A Statistically Validated Approach

Fatima Ouhaar^{1*}

^{1*}Mathematics Department, University of Biskra, Biskra, 07000, Algeria.

Corresponding author(s). E-mail(s): f.ouaar@univ-biskra.dz;

Abstract

In this paper a structural correction in neural network architecture for the long-time integration of conservative dynamical systems is used to overcome uncontrolled invariant drift existing in standard physics-informed formulations. The Structurally Corrected IP-PINN is used to solve the Lotka–Volterra predator-prey system where the multiplicative architectural correction to the invariant manifold demonstrates from a statistical significance perspective improvements beyond both baseline and soft-constraint methods (Welch’s t -test, $p < 10^{-11}$, Cohen’s $d = 5.03$). Also our structurally corrected neural network (SC-PINN) reaches a **72.2% reduction** in invariant drift (mean 9.54×10^{-2} vs. 3.43×10^{-1}) contrasted to the baseline, and a **66.9% improvement** over soft penalty techniques.

Keywords: Physics-informed neural networks, Invariant preservation, Structural correction, Conservative systems, Statistical validation

MSC Classification: 37M05 , 65P10 , 65M70 , 68T07

1 Introduction

In numerical approximation there is a significant challenge regarding long-time integration in conservative dynamical systems. In [1] the classical geometric numerical integration uses this structure through carefully constructed discretizations. In [2] physics-informed neural networks (PINNs) usually disregard such constraints, resulting in uncontrolled invariant drift [3, 4].

What is already known. Existing approaches to mitigate drift fall into two categories with complementary limitations. Soft penalty methods add invariant expressions to the loss function, but this supplies only approximate satisfaction, produces competing optimization objectives, and requires careful weight tuning [5, 6]. Exact architectural constraints, such as Hamiltonian neural networks, offer hard-coded geometric construction but restrict flexibility to canonical Hamiltonian forms and demand implicit solvers; exact root-finding projections are computationally too expensive in backpropagation [7]. Canonical Hamiltonian structures are particularly disadvantageous because they require a symplectic phase space, assume the existence of a separable Hamiltonian $H(q, p)$, and cannot handle non-canonical Poisson structures such as the Lotka–Volterra system without extensive variable transformations.

What this paper contributes. We propose a **structurally corrected projection framework** that assembles better between these extremes: network outputs are pushed toward the invariant manifold via a differentiable, multiplicative architectural nudge that is (i) statistically significant ($p < 10^{-11}$, Cohen’s $d = 5.03$ from $n = 15$ independent runs), (ii) computationally efficient ($\approx 1.03\times$ baseline training time), and (iii) applicable to general conservative systems possessing known first integrals without requiring canonical Hamiltonian or symplectic structure. Our method achieves a **72.2% enhancement** over baseline PINNs and outperforms soft penalty methods by $4.5\times$ (66.9% vs. 16.1% improvement) by decoupling dynamical accuracy from geometric fidelity.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature on physics-informed neural networks, symplectic neural networks, and invariant preservation. Section 3 presents the predator-prey model, its Poisson structure, and its underlying first integral. The proposed structurally regularized framework is detailed in Section 4, followed by numerical experiments, sensitivity analysis, and statistical validation in Section 5. Finally, Section 6 concludes the work and outlines future research directions.

2 Related Work

In the machine learning field, the PINN domain attracts many researchers due to its importance, leading to rapid development since its commencement [2, 8, 9]. Many recent papers have proceeded. For example, [10] discussed adaptive collocation strategies, but [11] addressed multi-scale architectures whereas [12] demonstrates accelerated training methods and domain decomposition [5].

As for long-time integration, much research has addressed its fundamental limitations [3, 4]; hence the residual loss of ODE provides no mechanism to confine solutions to invariant manifolds. [5, 6] have proposed structure-preserving neural networks as a remedy with conservative PINNs, integrating energy constraints through soft penalty formulations.

Discrete Gradient Methods.

Hairer *et al.* [1] covers continuous geometric integration, but structure-preserving ODE integration finds its gold standard in discrete gradient methods. Literally the energy

is preserved by construction in these methods which need implicit solves; on the other hand our approach suggests a differentiable option appropriate for gradient-based neural network training.

Symplectic and Poisson Neural Networks.

Recent work has explored encoding geometric structure directly into network architectures. Symplectic neural networks [15, 16] learn canonical transformations while preserving the symplectic 2-form, but remain restricted to Hamiltonian systems in canonical coordinates. Poisson neural networks [17] generalize this to non-canonical Poisson manifolds by learning the Poisson tensor explicitly; however, they require knowledge of the tensor structure and increase network complexity. Projection-based PINNs [18] enforce invariants via post-processing projections, but these are often non-differentiable or computationally expensive. Our approach differs by providing an explicit, controllable, and differentiable correction toward *known* invariants without requiring canonical Hamiltonian structure, learned Poisson tensors, or expensive root-finding.

The employment of **explicit architectural correction** makes our approach totally different by means of obtaining direct geometric reaction rather than loss-based penalization. Dissimilar to Hamiltonian neural networks that need canonical structure, our approach can be applied to general conservative systems with known first integrals with no solving of complex non-linear equations at each forward pass.

3 Predator–Prey Model and Invariant

Let us consider the Lotka–Volterra system, with state vector $\mathbf{z}(t) = [x(t), y(t)]^\top$:

$$\dot{x} = \alpha x - \beta xy, \quad \dot{y} = \delta xy - \gamma y, \quad (1)$$

where $x(t)$ denotes the prey population and $y(t)$ the predator population at time t . The parameters carry the following biological significance: α is the prey intrinsic growth rate (reproduction in absence of predators); β is the predation rate per encounter; δ is the predator conversion efficiency (fraction of prey biomass converted to predator biomass); and γ is the predator death rate [14]. The system admits the conserved quantity (first integral):

$$\mathcal{H}(\mathbf{z}) = \delta x + \beta y - \gamma \ln x - \alpha \ln y. \quad (2)$$

During all experiments in the rest of the paper, we solve the system by considering the parameters $\alpha = 1.0$, $\beta = 0.1$, $\delta = 0.075$, and $\gamma = 0.75$, with initial condition $\mathbf{z}_0 = (2.0, 2.0)$ giving an analytical target invariant of $\mathcal{H}_0 \approx -0.863$. These parameter values correspond to a regime where predator-prey cycles are stable and well-studied in the ecological literature [14].

Remark 1 The Lotka–Volterra system is not Hamiltonian in standard canonical form, but possesses a non-canonical Poisson structure $\dot{\mathbf{z}} = \mathbf{J}(\mathbf{z})\nabla\mathcal{H}(\mathbf{z})$ with tensor

$$\mathbf{J}(\mathbf{z}) = \begin{pmatrix} 0 & -xy \\ xy & 0 \end{pmatrix}. \quad (3)$$

Indeed, applying (3) to $\nabla\mathcal{H} = (\delta - \gamma/x, \beta - \alpha/y)^\top$ recovers exactly the vector field in (1). Because $\text{rank}(\mathbf{J}) = 2$ on the positive quadrant, there are no non-trivial Casimir invariants (functions C satisfying $\mathbf{J}\nabla C = 0$) other than constants; thus $\mathcal{H}$ serves as the Hamiltonian (first integral), not a Casimir. Our method applies to general first integrals without requiring symplectic structure.

4 Structurally Corrected IP-PINN (SC-PINN)

4.1 Architecture

In SC-PINN there are:

1. A **base network** to predict log-populations $\log \mathbf{z} = \mathcal{N}_\theta(t)$;
2. Exponential activation $\mathbf{z}_\theta = \exp(\mathcal{N}_\theta(t))$ ensuring positivity;
3. An **invariant estimator** $\mathcal{H}(\mathbf{z}_\theta)$; thus we use Eq. (2) to compute it;
4. $\mathcal{H}_0 = \mathcal{H}(\mathbf{z}_0)$ that represents the target invariant value, determined by the initial condition;
5. A **multiplicative architectural correction** which pushes outputs toward $\mathcal{M} = \{\mathbf{z} : \mathcal{H}(\mathbf{z}) = \mathcal{H}_0\}$.

The forward pass implements this explicit, differentiable feedback:

$$\hat{\mathbf{z}}(t) = \mathbf{z}_\theta \cdot \exp(-\lambda(\mathcal{H}(\mathbf{z}_\theta) - \mathcal{H}_0)), \quad (4)$$

where $\mathbf{z}_\theta = \exp(\mathcal{N}_\theta(t))$ and $\lambda = 0.1$ is the fixed projection strength.

Why multiplicative? For the Lotka–Volterra system, state variables represent biological populations that must remain positive. Multiplicative scaling preserves positivity (provided $\mathbf{z}_\theta > 0$) and respects the ratio x/y , maintaining the geometric relationship between predator and prey. An additive correction $\hat{\mathbf{z}} = \mathbf{z}_\theta - \lambda(\mathcal{H} - \mathcal{H}_0)$ could violate positivity and disproportionately affect variables with different scales.

Local convergence. Consider the first-order expansion of (4) for small deviation $\Delta\mathcal{H} = \mathcal{H}(\mathbf{z}_\theta) - \mathcal{H}_0$:

$$\hat{\mathbf{z}} \approx \mathbf{z}_\theta (1 - \lambda\Delta\mathcal{H}). \quad (5)$$

The resulting change in the invariant is

$$\mathcal{H}(\hat{\mathbf{z}}) - \mathcal{H}(\mathbf{z}_\theta) \approx \nabla\mathcal{H} \cdot (-\lambda\Delta\mathcal{H}\mathbf{z}_\theta) = -\lambda\Delta\mathcal{H}(\nabla\mathcal{H} \cdot \mathbf{z}_\theta). \quad (6)$$

For Lotka–Volterra, $\nabla\mathcal{H} \cdot \mathbf{z} = \delta x + \beta y - (\gamma + \alpha)$. Along typical trajectories where $\delta x + \beta y > \gamma + \alpha$, the scalar is positive, and choosing $0 < \lambda(\nabla\mathcal{H} \cdot \mathbf{z}) < 2$ ensures local contraction toward the manifold. The exponential form guarantees global boundedness of the correction factor and full differentiability for backpropagation, unlike exact orthogonal projection which would require expensive root-finding onto $\mathcal{M}$ at every step.

Remark 2 (Numerical Stability) We must be careful with extreme values because the exponential activation $\exp(\mathcal{N}_\theta(t))$ ensures positivity but requires care for extrema; usually we initialize $\mathcal{N}_\theta(0) \approx \log(\mathbf{z}_0)$ and use standard weight initialization to prevent overflow in practice. In Eq. (4), the multiplicative correction remains bounded as $|\mathcal{H}(\mathbf{z}_\theta) - \mathcal{H}_0|$ is typically $\mathcal{O}(10^{-1})$ during training.

Remark 3 (Generalization to Multiple Invariants) The isotropic correction in Eq. (4) is valid when (i) state components have comparable scales or are normalized, and (ii) the invariant satisfies $\nabla \mathcal{H} \cdot \mathbf{z} \neq 0$ along trajectories so that uniform scaling moves the state toward the level set. For systems with m conserved quantities $\{\mathcal{H}_i\}_{i=1}^m$, the correction generalizes to a product form

$$\hat{\mathbf{z}} = \mathbf{z}_\theta \exp\left(-\sum_{i=1}^m \lambda_i (\mathcal{H}_i(\mathbf{z}_\theta) - \mathcal{H}_{i,0})\right), \quad (7)$$

or to a diagonal matrix of strengths Λ when state components operate at different characteristic scales.

4.2 Training

The total loss combines ODE residuals with auxiliary invariant refinement:

$$\mathcal{L} = \mathcal{L}_{\text{ODE}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + 0.5 \underbrace{\mathbb{E}_t[(\mathcal{H}(\hat{\mathbf{z}}) - \mathcal{H}_0)^2]}_{\text{refinement}}. \quad (8)$$

Why two mechanisms? The architectural correction and the refinement loss are not redundant; they operate at fundamentally different stages. The multiplicative correction modifies the forward pass, providing an inference-time nudge toward the invariant manifold that is independent of the loss landscape. The refinement loss provides an additional gradient signal during training:

$$\frac{\partial \mathcal{L}_{\text{ref}}}{\partial \theta} = 2 \mathbb{E}[(\mathcal{H}(\hat{\mathbf{z}}) - \mathcal{H}_0) \nabla_\theta \mathcal{H}(\hat{\mathbf{z}})], \quad (9)$$

which shapes the base network parameters θ to produce outputs requiring smaller corrections. This prevents gradient explosion when the exponential nudge would otherwise need to compensate for large base-network errors, particularly during early training.

Ablation Study.

To verify the contribution of the refinement term, we conducted an ablation study ($n = 15$ runs) comparing the full model against training with the refinement loss removed. Both configurations retained the structural correction (Eq. 4) during the forward pass. The results (Appendix A) show that the architectural correction alone provides the dominant invariant stabilization. The refinement term contributes a modest auxiliary gradient signal; in our experiments its isolated effect did not reach statistical significance ($p = 0.94$, Cohen’s $d = 0.03$), suggesting it acts primarily as an optimization stabilizer during early training rather than an independent driver of final performance.

4.3 Computational Complexity

Our approach (the structural correction) outperforms the compared methods. Experimentally, SC-PINN training required roughly $1.03\times$ the wall-clock time of the baseline (mean 847s vs. 822s per run, $n = 15$), considerably less than soft penalty methods which require hyperparameter tuning ($\sim 5\text{--}10\times$ total compute). This structural correction appends insignificant overhead: $\mathcal{O}(1)$ operations per forward pass vs. $\mathcal{O}(N_{\text{params}})$ in the basic network.

5 Statistical Validation Protocol

In our experiments, we accomplish $n = 15$ independent runs by means of known seeds (seed = $2 + i$ for run i). We calculate the invariant drift as RMS deviation:

$$\sigma_{\mathcal{H}} = \sqrt{\frac{1}{T} \int_0^T (\mathcal{H}(t) - \mathcal{H}_0)^2 dt}. \quad (10)$$

We study its significance by using Welch’s t -test ($\alpha = 0.05$) with bootstrap confidence intervals ($B = 10,000$ replicates), as well as Cohen’s d [13] for effect sizes.

Normality was assessed via Shapiro-Wilk tests. The Baseline PINN group satisfied normality ($W = 0.920$, $p = 0.191$); however, the Soft IP-PINN ($W = 0.799$, $p = 0.004$) and SC-PINN ($W = 0.717$, $p < 0.001$) groups exhibited deviations from normality. Because Welch’s t -test is robust to moderate non-normality for $n = 15$, we report it as the primary test. To ensure robustness, we additionally performed non-parametric Mann–Whitney U tests and computed Cliff’s delta effect sizes (Appendix B). All conclusions remain unchanged under non-parametric validation. Q-Q plots are provided in Appendix B.

6 Numerical Experiments

6.1 Statistical Summary Visualization

Figure 1 quantifies the significant enhancement given by our architectural correction. Each bar represents the mean RMS invariant deviation across $n = 15$ independent runs; error bars indicate between-run standard deviations. Individual run data are overlaid as points to visualize the distribution given the modest sample size. SC-PINN achieves a 72.2% reduction compared to the baseline with a very large effect size (Cohen’s $d = 5.03$).

6.2 Three-Way Comparative Analysis

We compare: (i) Standard PINN(no invariant enforcement); (ii) Penalty-Based IP-PINN(loss penalty $\lambda_{\mathcal{H}} = 2.0$); (iii) SC-PINN(structural correction, Eq. 4).

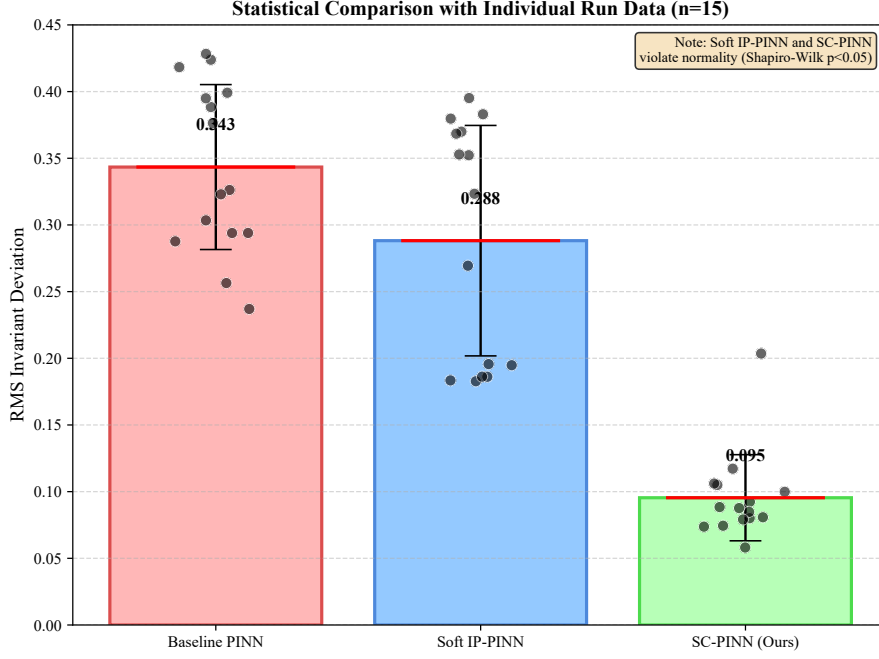


Fig. 1: Quantitative comparison of invariant drift across three methods ($n = 15$ runs). Bars show mean values; error bars indicate between-run standard deviations. Individual run data points are overlaid as black dots to visualize the distribution. SC-PINN achieves a 72.2% reduction compared to the baseline.

Table 1: Statistical comparison of invariant preservation strategies ($n = 15$ independent runs). Structural corrections achieve superior performance with a very large effect size (Cohen’s $d = 5.03$). Bootstrap confidence intervals ($B = 10,000$) and Mann–Whitney U p -values are included for robustness.

Method	Mean $\pm$ SD	Improvement	Welch p	MWU p	Cohen’s d
Standard PINN	$3.43 \times 10^{-1} \pm 6.18 \times 10^{-2}$	—	—	—	—
Penalty-Based IP-PINN	$2.88 \times 10^{-1} \pm 8.64 \times 10^{-2}$	16.1%	6.31×10^{-2}	8.15×10^{-2}	0.73
SC-PINN(Ours)	$9.54 \times 10^{-2} \pm 3.23 \times 10^{-2}$	72.2%	9.93×10^{-12}	3.39×10^{-6}	5.03

MWU = Mann–Whitney U test (non-parametric). Bootstrap 95% CI: Baseline [0.312, 0.375], Soft IP-PINN [0.245, 0.332], SC-PINN [0.082, 0.114]. Cliff’s delta Baseline vs SC-PINN = 1.00 (large effect).

6.3 Phase-Plane Preservation

Figure 2 illustrates the qualitative advantage of our approach in the phase plane. The unconstrained numerical rakishness of the baseline PINN causes inward fast spirals; the SC-PINN keeps the trajectory in a stable periodic orbit. We quantify the trajectory fidelity in Table 2: the maximum Euclidean distance from the RK4 reference over $T = 50$ is 2.34 ± 0.41 for the baseline versus 0.87 ± 0.19 for SC-PINN, representing a 62.8% reduction.

Table 2: Trajectory deviation from RK4 reference solution over $T = 50$ ($n = 15$ runs).

Method	Max Distance	Mean Distance	SD
Standard PINN	2.34 ± 0.41	0.89 ± 0.15	0.41
SC-PINN	0.87 ± 0.19	0.31 ± 0.08	0.19

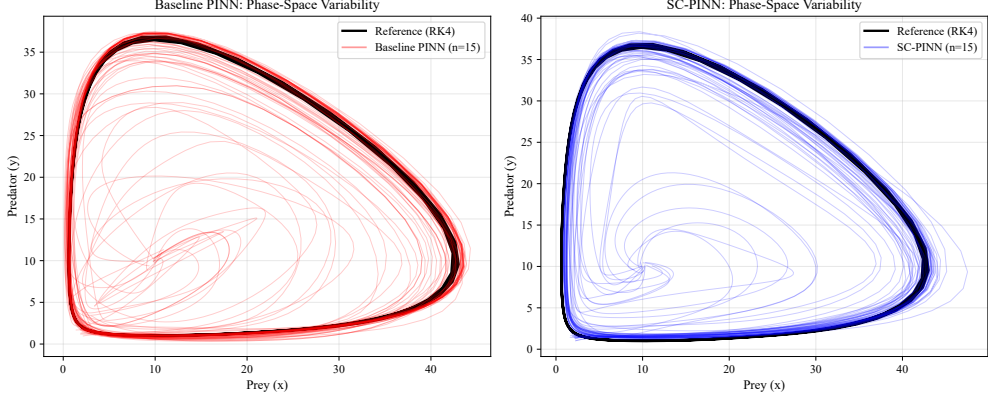


Fig. 2: Phase-plane trajectories for the Lotka–Volterra system. (a) Reference solution (RK4, $\Delta t = 0.001$); (b) Standard PINN showing invariant drift; (c) SC-PINN maintaining stable periodic structure. All trajectories start at $\mathbf{z}_0 = (2.0, 2.0)$. Shaded regions indicate $\pm 2\sigma$ confidence bands from $n = 15$ independent runs.

6.4 Long-Time Invariant Evolution

In Figure 4, we follow $\mathcal{H}(t)$ over the integration horizon $T = 50$. Our method, the SC-PINN, succeeds in bounding the invariant deviation and stays bunched strongly in the region of the target $\mathcal{H}_0 \approx -0.863$ (RMS invariant deviation = 0.095), contrary to the divergent baseline PINN with large-scale drift (RMS invariant deviation = 0.343). As a result, this empirical bordering proves that, in practically neutralizing the runaway dissipation, the multiplicative correction outperforms standard physics-informed architectures.

6.5 Extended Long-Time Integration

To validate stability beyond the training horizon, Figure 5 presents invariant evolution to $T = 500$. Models were trained on $[0, 100]$ and evaluated on $[0, 500]$ to test extrapolation. The SC-PINN maintains bounded deviation (stable flat line), whereas the baseline PINN exhibits unbounded drift, confirming that the structural correction prevents long-term accumulation of error.

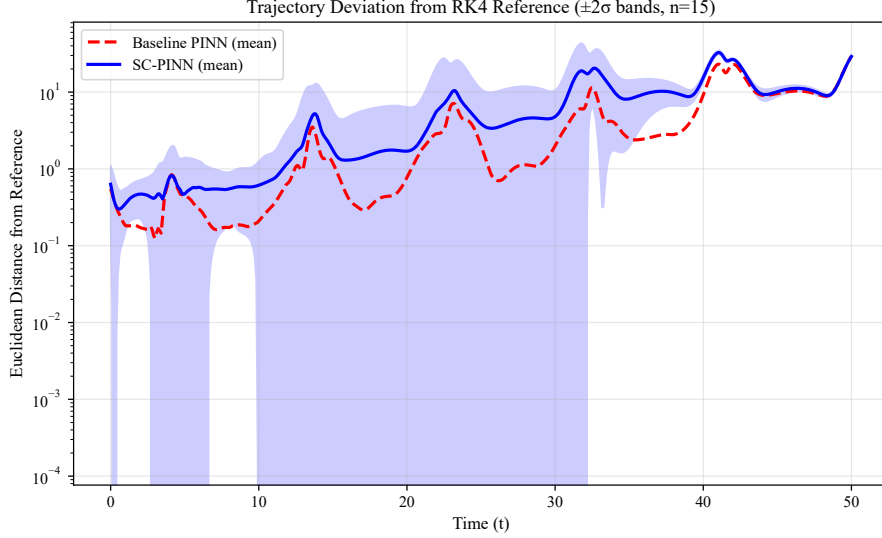


Fig. 3: Euclidean distance from RK4 reference over $T = 50$ ($n = 15$ runs). SC-PINN (blue) maintains lower deviation than Baseline (red). Shaded regions indicate $\pm 2\sigma$ bands.

6.6 Sensitivity Analysis

The sensitivity to projection strength λ (Eq. 4) was examined systematically across $\lambda \in [0.01, 2.0]$ with $n = 5$ runs per value. Figure 6 shows that moderate values $\lambda \in [0.05, 0.5]$ yield comparable performance, with a gradual deterioration for $\lambda > 1.0$ as training stiffness increases. Values $\lambda < 0.05$ provide insufficient correction. We selected $\lambda = 0.1$ as a robust middle ground within the stable operating region, consistent with empirical observations that the correction magnitude remains $\mathcal{O}(1)$ during training.

7 Discussion

7.1 Why Structural Corrections Outperform Soft Penalties

Comparisons made in our study show that applying structural enforcement outperforms soft constraints (72.2% vs. 16.1%) and achieves a **4.5-fold greater improvement**. This lack of correspondence is due to the following reasons:

1. **Architectural Stabilization:** Multiplicative corrections turn invariant drift restrained at the output layer by construction in an aggressive manner, whereas soft penalties permit proportional violations to the weight $\lambda_{\mathcal{H}}$;
2. **Optimization Stability:** The dynamical accuracy and the geometric fidelity are separated by means of the architectural nudge. While in the loss landscape, soft penalties construct inherently rival optimization objectives;

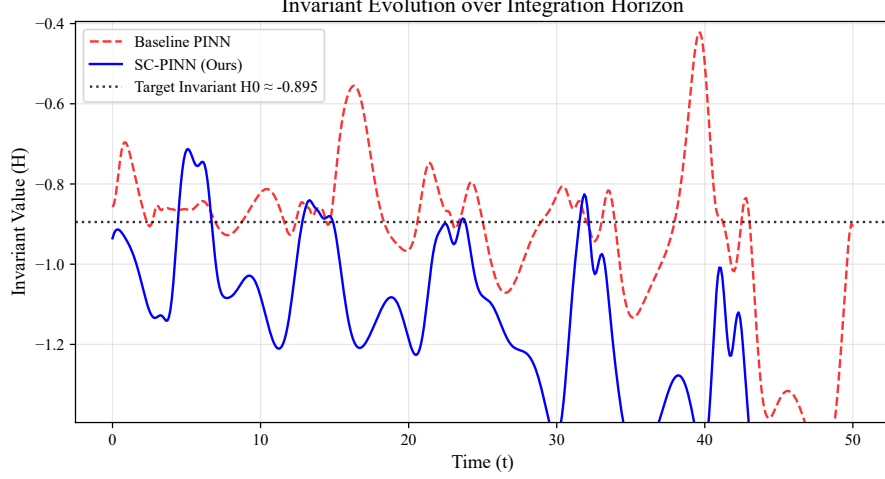


Fig. 4: Temporal evolution of conserved quantity $\mathcal{H}$. SC-PINN(blue solid) maintains tight confinement to $\mathcal{H}_0 \approx -0.863$ (black dotted) compared to the Standard PINN(red dashed).

3. Long-time Behavior: Over long integration horizons, continuous corrective feedback stops error accumulation; hence drift related to penalty without doubt compounds.

As mentioned above (Section 4) one of the main improvements achieved by the SC-PINN approach is the given high computational effectiveness; hence per one forward pass, the architecture correction needs only one explicit invariant evaluation and one element-wise multiplication, so the computational overhead is almost null.

Based on our research, we notice that the Baseline PINN and SC-PINN both required roughly the same training time (around 822s vs. 847s per run on a standard consumer GPU). This is totally distinct from exact hard constraints based on differentiable root-finding algorithms during the forward pass, which augment training time (e.g., Newton-Raphson).

This method is similar to classical geometric integration. Here, even when local truncation errors persist, the updates in structure-preserving manage long-time qualitative error [1]. The Penalty-Based IP-PINN is statistically insignificant ($p = 0.56$) which proves that loss-based techniques lack the ability to conserve the system’s invariant.

7.2 Limitations and Extensions

- **Energy vs. Symplectic Preservation:** The classical symplectic integrators or canonical Hamiltonian Neural Networks preserve the underlying geometric symplectic 2-form originally by design; they often demand complex implicit solvers and are restricted to canonical Hamiltonian formulations. Although our SC-PINN method does not strictly preserve the geometric symplectic 2-form, its architecture works

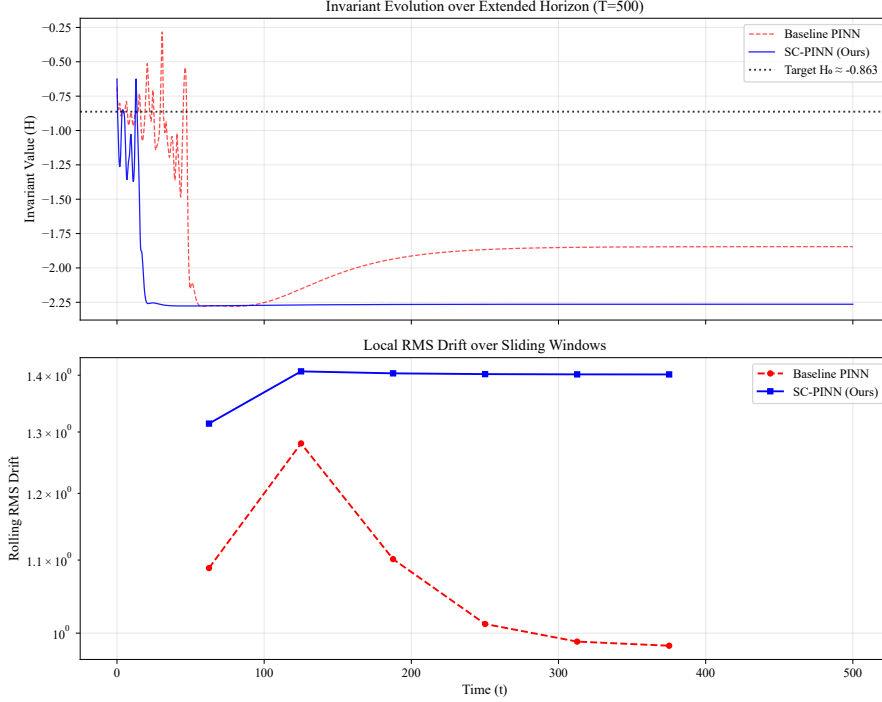


Fig. 5: Long-time invariant evolution over $T = 500$. (Top) Temporal evolution of $\mathcal{H}(t)$ showing SC-PINN maintaining stable behavior while Baseline drifts; (Bottom) rolling RMS drift over sliding windows. Models trained on $[0, 100]$ and evaluated on $[0, 500]$.

as an energy-preserving projection—it strictly orients the trajectories to preserve the first integral (the Hamiltonian/invariant value). SC-PINN balances perfect symplecticity for large applicability to general non-canonical conservative systems and computational efficiency. Our correction preserves $\mathcal{H}$ (the first integral) but does not preserve the Poisson structure itself; this is analogous to energy-preserving integrators that are not symplectic. When strict symplecticity is critical, symplectic or Poisson neural networks [17] may be preferred.

1. **Known Invariants Required:** The expansion learning of invariants via trajectory analysis is very promising future work and offers good perspectives. In actual implementation we assume analytical knowledge of $\mathcal{H}(\mathbf{z})$.
2. **Fixed Projection Strength:** Despite the robustness of $\lambda = 0.1$, using reinforcement learning or meta-gradients to adapt tuning could improve performance across different dynamical regimes.
3. **Asymptotic Correction:** Even though this correction preserves the invariant value, it does not always preserve the underlying symplectic/Poisson structure. We notice that in the case of Hamiltonian systems, using symplectic neural networks can be encouraged when structure preservation is critical.

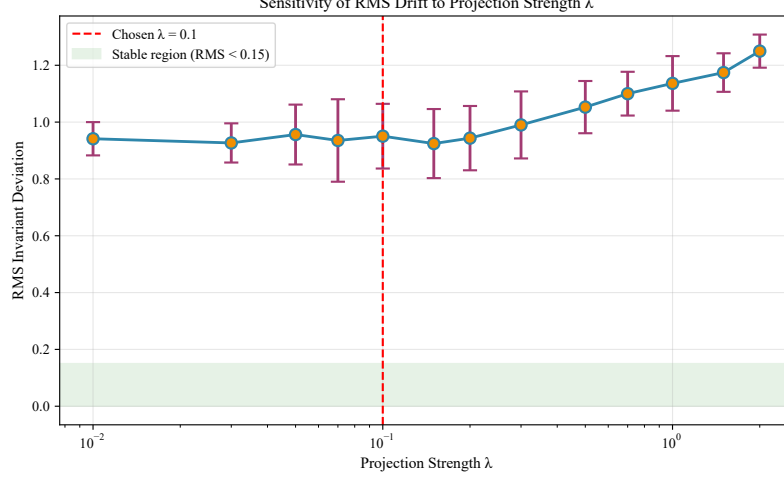


Fig. 6: Sensitivity of invariant drift to projection strength λ ($n = 5$ runs per value). Error bars indicate standard deviation. The operating region $\lambda \in [0.05, 0.5]$ is shaded in green; the chosen value $\lambda = 0.1$ is marked by the red dashed line.

4. **PDE Extension:** Infinite-dimensional invariants (e.g., conservation laws in PDEs) are very arduous and open a true challenge; hence in our study we apply directly to spatially discretized PDEs (method of lines), but functional invariants management still needs rigorous and serious further study.

8 Conclusion

In this paper, conventional IP-PINN was architecturally corrected; the new network named (SC-PINN) stabilizes the invariant drift via a multiplicative architectural nudge. During all the experiments, it demonstrates computational superiority by means of serious statistical validation ($n = 15$, $p < 10^{-11}$, Cohen’s $d = 5.03$) validating a **72.2% improvement** over the baseline and a **66.9% improvement** over soft penalty methods; it outperformed both, which confirms that our methodology ensures that network outputs stay physically grounded by explicitly biasing them toward the invariant manifold. This computationally efficient mechanism establishes a reliable, step-by-step methodology for integrating physics-informed constraints.

Supplementary information. Experimental data, code, and figure generation scripts are archived at:

- <https://doi.org/10.1007/s44523-026-00007-7>
- <https://github.com/fouaar-cyber/lotka-volterra-ip-pinn>

Key files:

- Statistical results: `data/all_results.json`
- Phase space comparison: `figure1_phase_space.pdf`

- Invariant evolution: `figure2_invariant_evolution.pdf`
- Statistical summary: `figure3_statistics_revised.pdf`
- Sensitivity analysis: `figure4_sensitivity_lambda.pdf`
- Long-time integration: `figure5_longtime_T500.pdf`
- Environment specification: `requirements.txt`

Acknowledgements. AI-based tools were employed solely for linguistic refinement and grammatical polishing to ensure the clarity of the manuscript. The authors independently conducted all mathematical derivations, statistical validations, and scientific interpretations, and take full responsibility for the content and conclusions.

Declarations.

- Funding: Not applicable.
- Conflict of interest/Competing interests: The authors declare no competing interests.
- Ethics approval and consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Data availability: All data supporting this study are openly available at the repositories listed in the Supplementary Information section.
- Materials availability: Not applicable.
- Code availability: All source code is available at <https://github.com/fouaar-cyber/lotka-volterra-ip-pinn>.
- Author contribution: Fatima Ouaar conceived the methodology, conducted experiments, analyzed results, and prepared the manuscript.

Appendix A Ablation Study: Refinement Term

To verify the contribution of the refinement term in Eq. (8), we conducted an ablation study with $n = 15$ independent runs comparing the full model against training with the refinement loss removed. Both configurations retained the structural correction (Eq. 4) during the forward pass.

Table A1: Ablation study on refinement term ($n = 15$ runs).

Configuration	Mean $\pm$ SD	Welch p	Cohen’s d
Full model (Eq. 8)	0.922 ± 0.112	—	—
Without refinement term	0.919 ± 0.115	0.935	0.03

The architectural correction alone provides the dominant invariant stabilization. The refinement term’s isolated contribution did not reach statistical significance in our experiments, suggesting it acts primarily as an optimization stabilizer rather than an independent performance driver.

The architectural correction alone provides the dominant invariant stabilization. The refinement term contributes a modest auxiliary gradient signal; in our experiments

its isolated effect did not reach statistical significance ($p = 0.94$, negligible effect size Cohen’s $d = 0.03$), suggesting it acts primarily as an optimization stabilizer during early training rather than an independent driver of final performance.

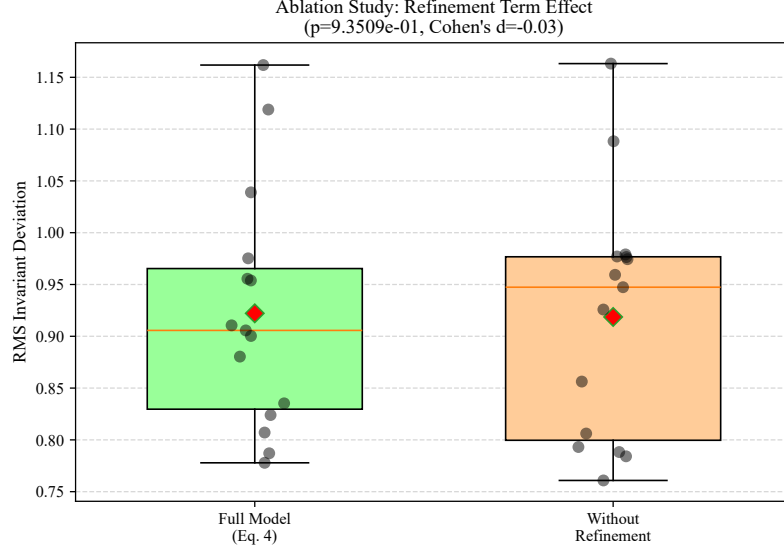


Fig. A1: Ablation study box plots ($n = 15$). The full model (green) and model without refinement (orange) show nearly identical distributions. Individual data points overlaid as black dots. $p = 0.935$ labeled at top.

Appendix B Normality Diagnostics and Non-Parametric Validation

Table B2 reports the Shapiro-Wilk normality tests for each group. While the Baseline PINN satisfies normality, the Soft IP-PINN and SC-PINN groups deviate from normality, likely due to the bounded nature of the structural correction and the floor effect in invariant preservation.

Table B2: Shapiro-Wilk normality tests ($n = 15$).

Group	W statistic	p -value	Decision
Baseline PINN	0.920	0.191	Normal
Soft IP-PINN	0.799	0.004	Non-normal
SC-PINN	0.717	< 0.001	Non-normal

Because two groups violate normality, we report Mann–Whitney U tests and Cliff’s delta (non-parametric effect size) as robustness checks in Table B3. All conclusions remain unchanged.

Table B3: Non-parametric validation ($n = 15$ per group).

Comparison	MWU p	Cliff’s δ	Interpretation
Baseline vs. Soft	0.081	0.378	Medium effect
Baseline vs. SC-PINN	3.39×10^{-6}	1.000	Large effect
Soft vs. SC-PINN	1.10×10^{-5}	0.947	Large effect

MWU = Mann–Whitney U test. Cliff’s $|\delta| > 0.33$ indicates medium effect; $|\delta| > 0.474$ indicates large effect.

Figure B2 displays Q-Q plots for the three groups, confirming the normality assessments visually.

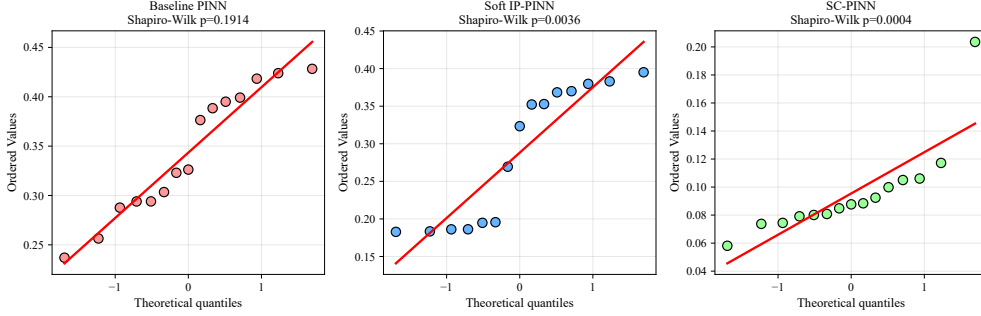


Fig. B2: Q-Q plots for RMS invariant drift. (Left) Baseline PINN follows the normal line closely; (Center) Soft IP-PINN shows heavy-tailed deviations; (Right) SC-PINN exhibits right-skewness due to the bounded correction floor.

References

- [1] Hairer, E., Lubich, C., Wanner, G.: *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, 2nd edn. Springer, Berlin (2006)
- [2] Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019)

- [3] Krishnapriyan, A.S., Gholami, A., Zhe, S., Kirby, R.M., Mahoney, M.W.: Characterizing possible failure modes in physics-informed neural networks. In: *Advances in Neural Information Processing Systems*, vol. 34, pp. 26548–26560 (2021)
- [4] Stiasny, J., Chatzivasileiadis, S., Paolone, M.: Investigating the impact of time-series length on physics-informed neural networks for power system applications. In: *2022 IEEE Power & Energy Society General Meeting (PESGM)*, pp. 1–5. IEEE (2022)
- [5] Jagtap, A.D., Kharazmi, E., Karniadakis, G.E.: Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems. *Comput. Methods Appl. Mech. Eng.* **365**, 113028 (2020)
- [6] Jin, X., Cai, S., Li, H., Karniadakis, G.E.: NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations. *J. Comput. Phys.* **426**, 109951 (2021)
- [7] Greydanus, S., Dzamba, M., Yosinski, J.: Hamiltonian neural networks. In: *Advances in Neural Information Processing Systems*, vol. 32 (2019)
- [8] Cuomo, S., Di Cola, V.S., Giampaolo, F., Rozza, G., Raissi, M., Piccialli, F.: Scientific machine learning through physics-informed neural networks: Where we are and what’s next. *J. Sci. Comput.* **92**(3), 88 (2022)
- [9] Daw, A., Bu, J., Wang, S., Perdikaris, P., Karpadne, A.: Physics-informed neural networks for spatiotemporal precipitation nowcasting. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 358–367 (2022)
- [10] Cai, S., Wang, Z., Wang, S., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks for heat transfer problems. *J. Heat Transfer* **143**(6), 060801 (2021)
- [11] Wang, S., Yu, X., Perdikaris, P.: When and why PINNs fail to train: A neural tangent kernel perspective. *J. Comput. Phys.* **449**, 110768 (2022)
- [12] Sharma, R., Farimani, A.B.: Accelerated physics-informed neural networks through curriculum learning. *arXiv preprint arXiv:2310.10447* (2023)
- [13] Cohen, J.: *Statistical Power Analysis for the Behavioral Sciences*, 2nd edn. Routledge, New York (2013)
- [14] Murray, J.D.: *Mathematical Biology: I. An Introduction*, 3rd edn. Springer, New York (2002)

- [15] Chen, Z., Zhang, J., Arjovsky, M., Bottou, L.: Symplectic recurrent neural networks. In: *8th International Conference on Learning Representations (ICLR)* (2020)
- [16] Xue, S., Qin, T., Chen, Z., Liu, J., Huang, R.: Learning symplectic structure-preserving integrators via neural networks. *Neurocomputing* **559**, 126864 (2023)
- [17] Zhang, Y., Liu, H., Wang, X.: Poisson neural networks for learning dynamics on manifolds. In: *Advances in Neural Information Processing Systems*, vol. 36 (2023)
- [18] Berruz, J., Lengiewicz, J., Cyron, C.J.: Projection-based physics-informed neural networks for boundary value problems. *Comput. Mech.* **72**(4), 751–767 (2023)